



ARCHITECTURE · EFFICIENCY · TRAINING STABILITY · PROVENANCE

Qwen3.8-Flash-Next

架构、效率与技术谱系全景解读

从 $3 \times \text{GDN} + 1 \times \text{QSA}$ 的混合 token mixer，到四分支 Gated Residual、离卡 n-gram memory、Muon/Canzona 与稳定性压力测试：沿技术报告原有论证顺序，拆解这套 Qwen4 前瞻架构。

125B / 6B active

+51B n-gram

48 layers · 512 experts

256K native · 1M YaRN

00 · EXECUTIVE READING

核心判断：这不是四个孤立模块，而是一套耦合设计

Qwen3.8-Flash-Next 的主线可以概括为：用 GDN 压低大多数层的序列状态成本，用 QSA 保留周期性的精确全局检索；用四分支 GR 改写跨层信息流并提供显式幅度调节；再以离卡 n-gram 表扩充“记忆容量”，最后重新拟合 Muon 下的批量、学习率与稳定性边界。

TOKEN MIXING

常态压缩，周期检索

36 层 GDN 把历史压进定长状态；12 层 QSA 仍能直接访问被索引器选中的远端 token。

RESIDUAL

四条流，各司其职

GR 不再让所有层共享一条残差河道；实验显示一条分支形成长程通路，另外三条偏局部。

MEMORY

把容量移出算力核心

51B n-gram 参数以确定性地址访问，可放主机内存并预取；增加容量几乎不增加矩阵 FLOPs。

OPTIMIZATION

稳定性换取更激进配方

GR 的显式 gate 降低梯度与激活异常，使更大 batch、更高学习率和取消 batch warmup 成为可行选择。

“Architecture, efficiency, and optimization form one coupled system.”

Qwen3.8-Next Technical Report, § 5

报告最有价值的地方

它没有只给最终结构，而是系统展示“看似更省”的方案为什么被放弃：NoPE 在预训练阶段看不出问题，却在后训练后更易无休止生成；GR Top-2 稀疏读几乎不伤预训练指标，却损害后训练质量；batch warmup 不仅无收益，还增加 18.8% optimizer steps。这些反例比单独的 SOTA 表更能解释最终架构。

本文使用的证据等级

明确采用

报告直接写明 follows、adopts、based on，或官方配置逐项确认。

基于既有工作再设计

上游路线明确，Qwen 对门控、粒度、训练或系统实现做了关键修改。

相关但非直接祖先

解决相近问题或结构可比较，报告没有确认来源关系。

分析推断

由公式、配置或算子约束推得，明确与作者原结论分开。

01 · SCOPE & RELEASE AUDIT

报告范围与公开模型不是完全同一口径

技术报告主要研究文本主干的架构选择、效率和预训练稳定性；公开的 Qwen3.8-Flash-Next 权重则是一个已后训练的多模态模型。理解这一点，才能正确归属论文表格、官方模型卡 benchmark 和部署信息。

TECHNICAL REPORT

研究对象：文本主干与 base model

核心是 GDN、QSA、GR、n-gram embedding、Muon 与 scaling recipe。论文最终 Table 11 比较的是三款 Base 模型的 14 项预训练 benchmark。

OFFICIAL RELEASE

交付对象：多模态后训练模型

官方模型卡额外披露 27 层视觉编码器、4B MTP、语言与视觉 agent benchmark、思考模式以及 SGLang/vLLM/TokenSpeed 部署入口。

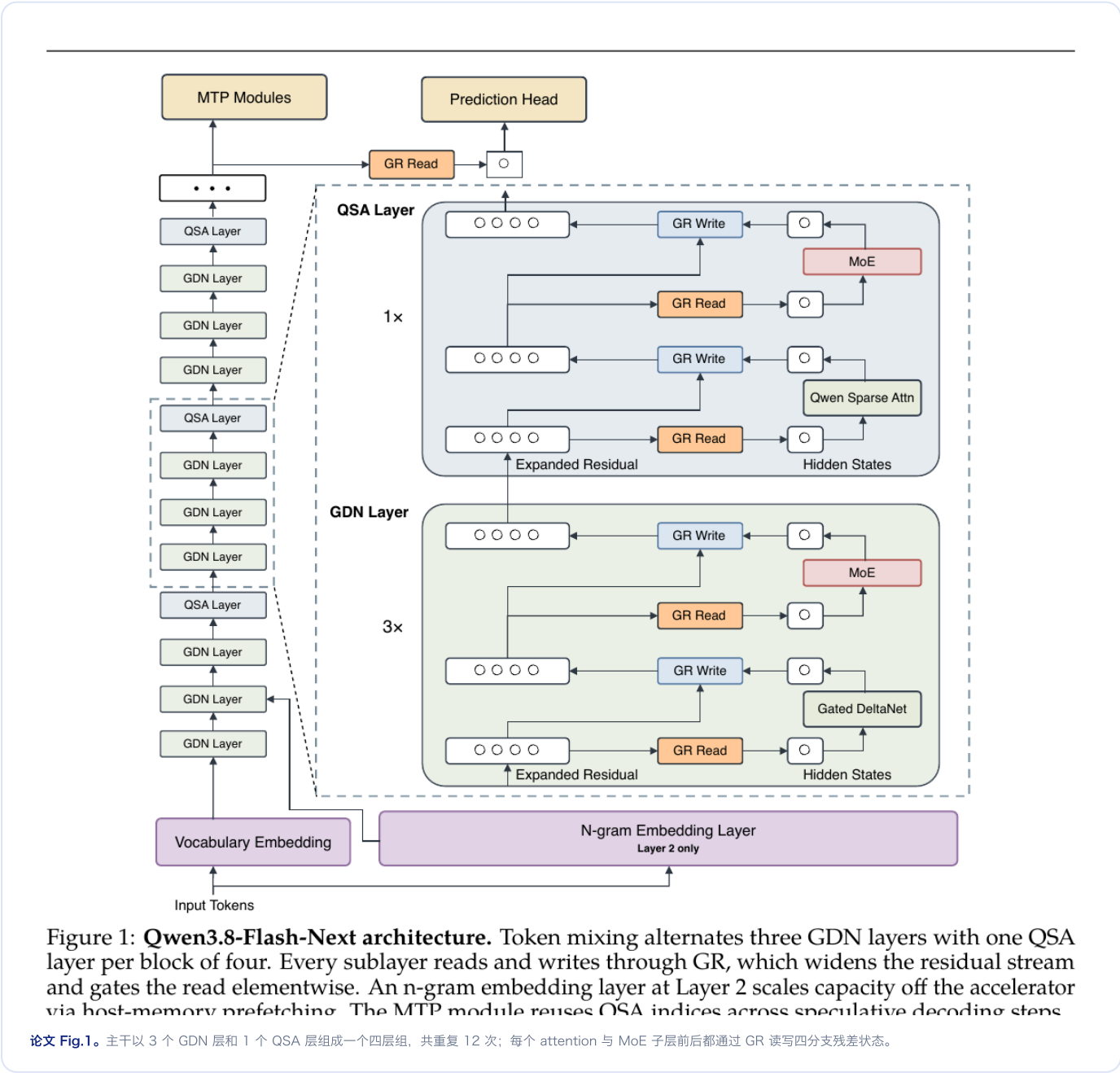
口径	公开信息	证据来源
语言主干	125B 总参数、每 token 激活 6B，另有 51B n-gram	技术报告 + 官方模型卡
MTP	1 层、约 4B 参数；训练为多步预测，QSA 索引在推测步骤间复用	模型卡 + 报告 Fig.1 / Table 4
多模态	27 层视觉编码器，输出维度 2560，接入同一语言主干	官方 config.json
上下文	原生 262,144；官方称可用 YaRN 扩展至 1,000,000	配置 + 官方博客
权重体积与协议	HF 仓库约 360GB，Qwen Community License 1.0	官方 HF 仓库

参数总数为什么会看到 125B、176B、4B 等不同数字

125B 是主语言模型参数；51B 是离卡 n-gram 表；4B 是 MTP。把三者相加可得到约 180B 的功能模块总量，但论文的主模型比较仍使用 125B/6B 口径。社区出现的“176B”通常只把 125B 与 51B 相加，没有计入 MTP。

02 · MACRO ARCHITECTURE

总体架构：48 层、512 专家、四条残差流



模块	官方配置	结构含义
主干	48 layers, hidden size 2560	12 组混合 token-mixing block
GDN	48 value heads / 16 QK heads, head dim 128, 短卷积核 4	36 层定长 recurrent state; Q/K/V 先经短因果卷积
QSA core	24 Q heads / 2 KV heads, head dim 256, RoPE dim 64	12 层稀疏全局检索; GQA core 降低 KV cache
QSA indexer	4 Q heads / 1 shared K, dim 128; K=2048, r=4	每个 query 最多选 512 个完整 micro-block, 再展开为 token
MoE	512 routed experts; Top-10 + 1 shared; expert dim 640	极高总容量、较低每 token 激活参数
GR	4 branches; low-rank 320=d/8	逐通道动态读 gate + 每分支标量写 gate
N-gram	layer 2; 20M slots; bigram/trigram; 128-way split	确定性 lookup memory, 可与第 1 层计算重叠预取

一条 token 在一层里经历什么



这意味着 GR 不是偶尔插入的附加层，而是每个 token mixer 和每个 MoE 的统一接口。它同时承担旧式 pre-norm 的归一化职责、分支选择职责和幅度控制职责，因此训练稳定性提升会反馈到 attention、MoE、学习率与 batch size 的选择。

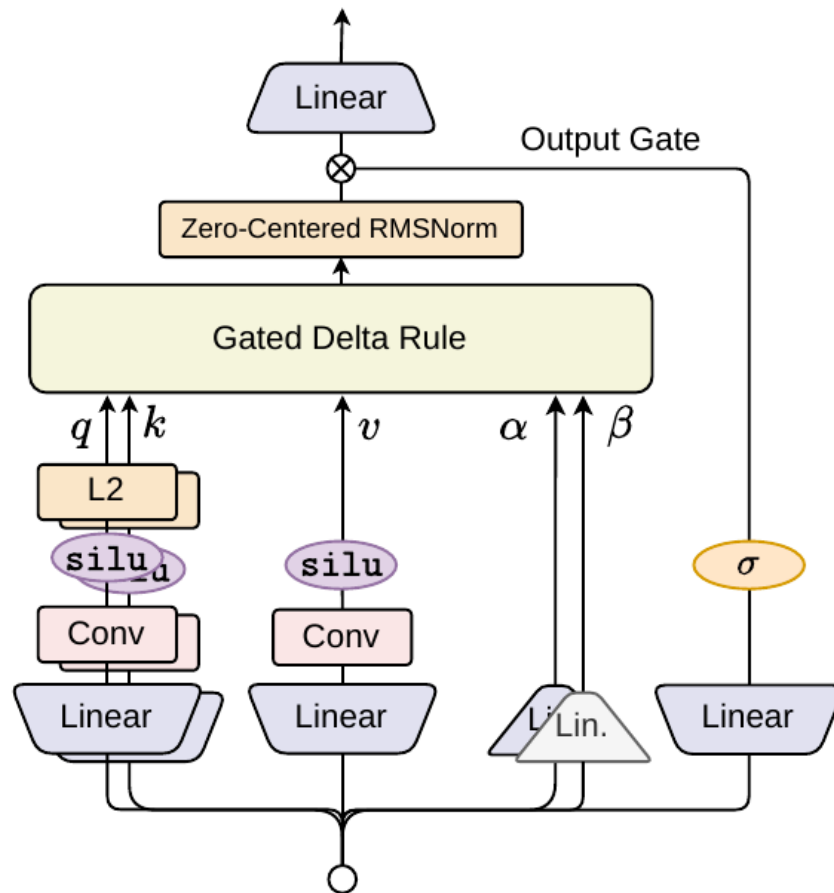
03 · GATED DELTANET

GDN：把绝大多数历史压进可更新的矩阵记忆

GDN 不是“缩小窗口的 attention”。它给每个 head 维护一个固定大小的 fast-weight 矩阵，当前 token 用 key 定位旧关联，用 value 与旧读出之差做增量写入，同时通过 decay gate 控制整张记忆的寿命。

tion

N Hybrid Architecture



The Gated DeltaNet token mixer. The projected query, key, and value streams parallel convolutions; queries and keys are L2-normalized before the gated delta recurrent update. α_t and write gate β_t control the recurrent update, while a sigmoid output gate moderates RMS-normalized output.

论文 Fig.2。Q/K/V 经过投影和短因果卷积；Q/K 做 L2 normalize；输出先做 zero-centered RMSNorm，再乘输入相关的 sigmoid gate。

$$\tilde{S}_{t-1} = \alpha_t S_{t-1}$$

$$e_t = v_t - \tilde{S}_{t-1}^T k_t$$

$$S_t = \tilde{S}_{t-1} + \beta_t k_t e_t^T$$

$$y_t = S_t^T q_t$$

DECAY A

全局遗忘

$\alpha \in (0,1)$ 衰减旧状态，控制已有记忆保留多久。

ERROR E

先读再比较

不是直接写 v，而是计算当前 key 已经读出了什么，再写残差误差。

WRITE B

定向改写

$\beta \in (0,1)$ 控制 rank-1 delta update 的强度，避免相似 key 无界累加。

Qwen 在原始 GDN 上改了什么

- 输出 gate 从 SiLU 改为 sigmoid。报告把这项改动明确写为相对原始 GDN 的差异；bounded positive gate 与 GR、gated attention 的稳定性观察一致。
- 统一使用 zero-centered RMSNorm。这一做法沿用 Qwen3-Next，用来限制 RMSNorm gain 漂移。
- 每四层保留一层全局 attention。有限状态记忆无法精确替代任意 token 的直接寻址，因此 QSA 周期性提供全局检索。
- 保留 RoPE。NoPE 的预训练指标相近，但后训练后无休止生成率明显更高；最终配置不采用 NoPE。

28-layer 25B-A3B; 400B@4K + 80B@32K	平均分	相对观察
Full attention	49.87	直接全局访问，成本最高
SWA hybrid (window 128)	51.15	局部高效，但窗口外只能靠层深传播
GDN hybrid	53.81	9 项中胜 full attention 8 项、胜 SWA 7 项

效率证据的正确范围

FlashQLA 在 NVIDIA GPU 上相对 FLA Triton kernel，forward 快 2 - 3×、backward 约 2×。这是 fused linear-attention kernel 的对比，不等同于完整模型端到端训练快 2 - 3×。

04 · QWEN SPARSE ATTENTION

QSA：不只稀疏 core attention，也压缩索引器本身

QSA 明确沿着 DeepSeek Sparse Attention 的“轻量 indexer → Top-K → sparse core attention”路线继续设计。它针对 DSA 的剩余瓶颈做了关键改动：先把 key 序列按 4-token micro-block 压缩，再在 block 级打分，将索引复杂度从 $O(n^2)$ 降为 $O(n^2/r)$ 。

computational bottleneck of softmax attention in long-context scenarios. Recently, DSA (Liu et al., 2023a) has achieved considerable inference speedups using a lightweight indexer to generate token-level sparse masks. However, as sequence length increases, the overhead of its $O(n^2)$ indexer remains non-negligible.

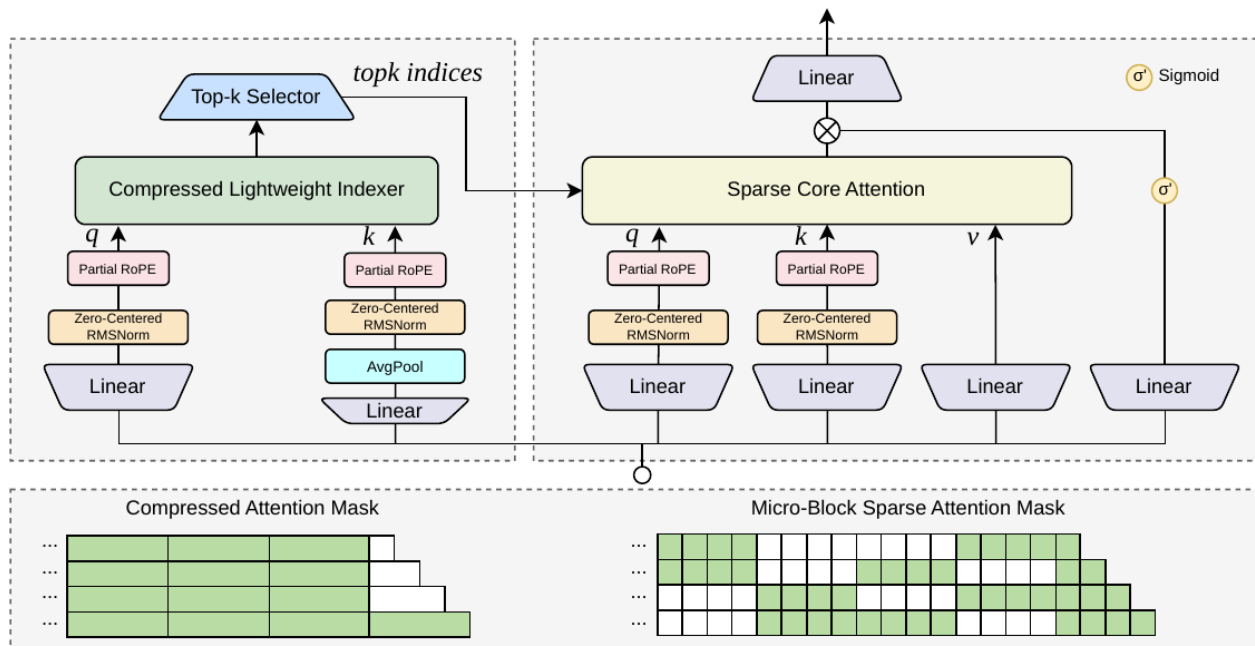


Figure 3: Overview of Qwen Sparse Attention (QSA). The QSA indexer (left) uses a compressed causal attention mask to score key blocks and select the top- k indices. These indices are expanded into a micro-block sparse attention mask for sparse core attention (right).

论文 Fig.3。左侧 indexer 用压缩 causal mask 给 key block 打分；选出的 block 展开为右侧 micro-block sparse mask，供 GQA core attention 计算。

Indexer 的具体数据流



$$\bar{k}_b = \text{RMSNorm}(\text{AvgPool}(k_{p \dots p+r-1}))$$

$$l_{ib} = \sum_{h=1 \dots H} \text{ReLU}(\langle q_{ih}, \bar{k}_b \rangle), \text{ 仅当该 block 已完整出现}$$

$$K_B = \lceil K/r \rceil = 512, K=2048, r=4$$

为什么池化必须发生在 RoPE 之前？ 如果先旋转再平均，相同内容会因 token 位置不同带着不同相位，平均后同时混合内容与旋转相位。QSA 先形成 block 内容摘要，再把该摘要放到 block 起始位置；这样一个 compressed key 对应一个清晰位置。

为什么尾部 incomplete block 永远加入？ 当前 query 附近的最新 token 还没凑满 4 个，不能进入只允许完整 block 的 causal indexer。直接把尾部 token 加入候选集，既保持严格因果性，也避免丢掉最邻近上下文。

两阶段训练：先学“去哪找”，再让主干适应“只看这些”

```
# Stage 1: dense distillation; backbone teacher 提供完整 attention
teacher_token = l1_normalize(sum_over_heads(softmax(full_attn_logits)))
teacher_block = l1_normalize(max_pool(teacher_token, block_size=4))
loss = KL(teacher_block || softmax(indexer_score))
train(indexer_only, steps=1000, lr=1e-3, tokens≈2B)

# Stage 2: sparse CPT; Top-K 真正进入主干计算
blocks = topk(indexer_score, 512)
tokens = expand(blocks) + causal_tail
output = sparse_core_attention(q, k[tokens], v[tokens])
train(backbone_and_indexer, steps=8000, lr=2.5e-5, tokens≈200B)
```

Stage 1 的 teacher block 用 **max pooling**，不是平均池化。原因是 attention 的关键证据可能集中在 block 内单个 token；平均会把尖峰稀释。Stage 2 则只在已选 block 上重新归一化 teacher probability，再计算 KL，让 indexer 与被稀疏化的主干一起适应。

QSA 结果	Full attention	QSA	解释
8 项常规 benchmark 平均	75.9	76.8	7/8 不低于 full attention；不出现系统性短上下文退化
RULER 512K - 1M	90.08	93.00	长上下文阶段反而更高
MRCR 512K / 1M	30.66 / 20.71	40.53 / 26.44	稀疏路由可能过滤大量无关上下文
四步 MTP accepted length	4.06	4.07	复用 Top-K index 基本不影响接受长度

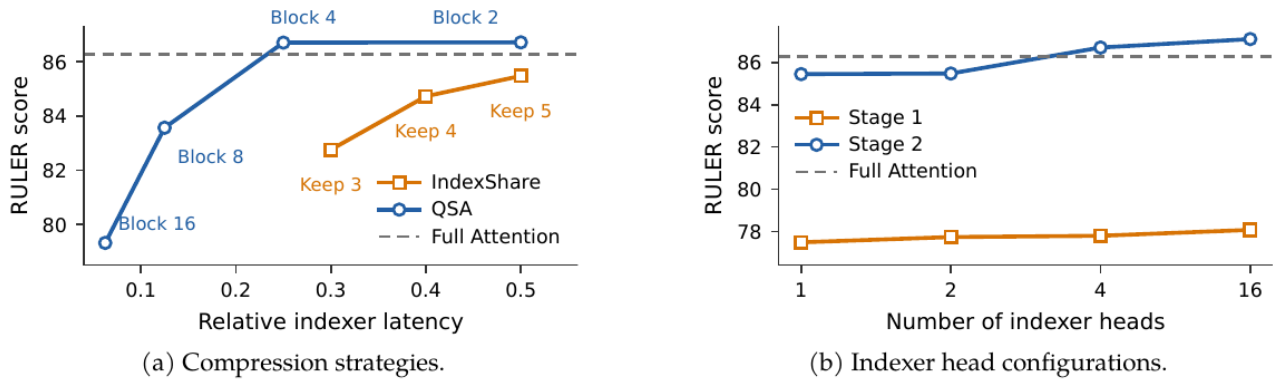


Figure 5: Architecture ablations of QSA on RULER. (a) QSA performance with different micro-block sizes; “Keep x ” indicates the number of IndexShare indexer layers retained for computation. (b) Performance with different numbers of indexer query heads after dense distillation and sparse training.

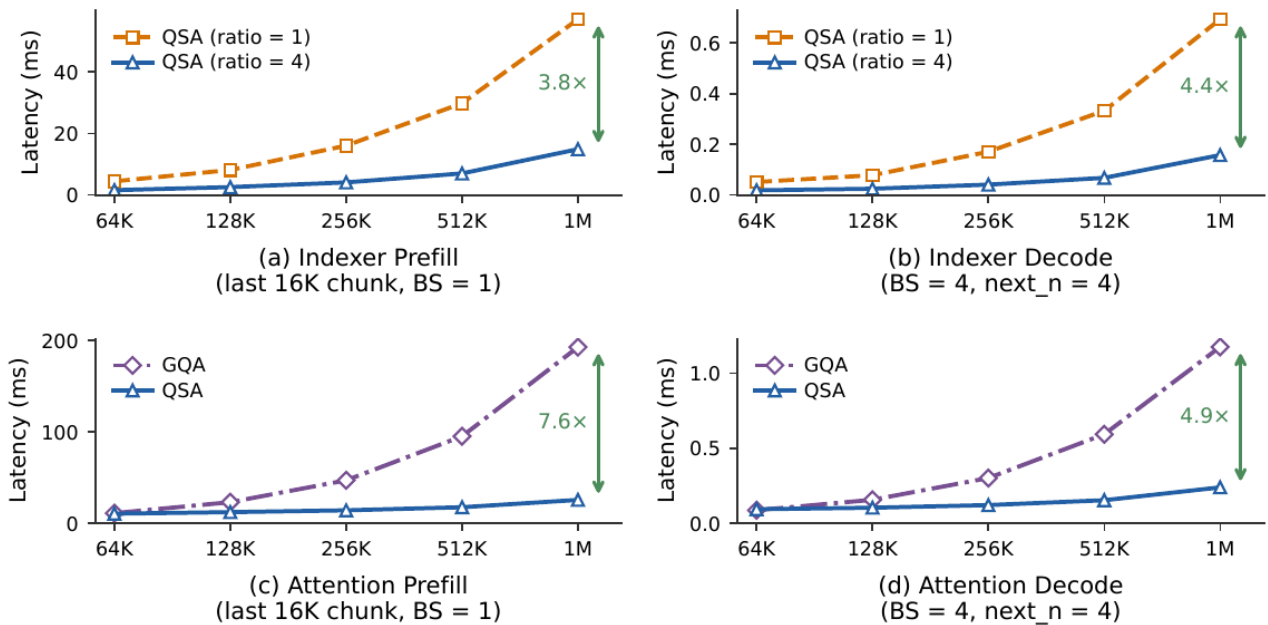


Figure 6: Kernel-level latency of QSA across context lengths during prefill and decode. Panels (a,b) compare indexer latency under different compression ratios, while panels (c,d) compare kernel-level attention latency between dense GQA and QSA, including both the indexer and sparse core attention. Chunked prefill uses a 16K-token chunk with batch size 1; decode uses batch size 4 and next_n = 4, corresponding to three MTP prediction steps. Arrows indicate speedups at a context length of 1M.

论文 Fig.5 - 6。micro-block=4 时 indexer latency 约为未压缩的 1/4；1M 上下文下，indexer prefill/decode 分别 3.8 $\times$ /4.4 $\times$ ，完整 attention module 相对 paged GQA 分别 7.6 $\times$ /4.9 $\times$ 。

为什么 QSA 适合 GDN/QSA 混合层

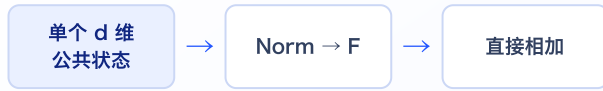
跨层共享 index 的方法依赖相邻全局 attention 层拥有相似重要 token，但这里两层 QSA 之间隔着三层 GDN，隐藏状态已被 recurrent mixer 改写。QSA 在本层内部压缩 key，不依赖跨层相似度；论文消融中，IndexShare 即使把索引成本降到 0.5，仍未恢复 full-attention 水平，而 QSA 在 0.25 达到基线。

普通残差、mHC、Attention Residual 与 GR：四种“跨层信息系统”

残差连接不只是让梯度绕过子层，它还规定了网络如何沿深度保存、读取和更新信息。四种方案可以用三个问题统一理解：历史被保存成什么状态？当前子层从哪里读？子层输出又写到哪里？

PRE-NORM RESIDUAL

单流累加：所有历史先压成一个和



读固定为当前总和，写固定为系数 1 的加法。状态最省、identity path 最清晰，但某个早期输出一旦被加进总和，后层便无法把它与其他历史输出分开取回。

MHC · FIXED-WIDTH MEMORY

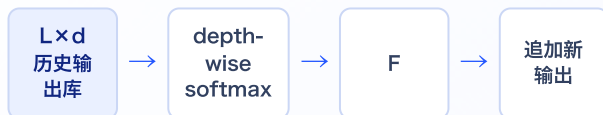
多流路由：标量读写 + 分支间动态混合



把历史压进固定数量的并行状态；每个 token 可改变各分支的读写权重，并用 H_{res} 在分支间搬运已有内容。mHC 将 H_{res} 约束为双随机矩阵，以限制动态混合的放大与漂移。

ATTENTION RESIDUAL · DEPTH MEMORY

历史库检索：直接 attention 到早期子层输出



每个子层不必接受“已经混成一团”的总和，而是对全部早期输出重新加权。检索最直接，但 Full AttnRes 保存和通信的状态随深度增长；Block 版用分块求和降低增长速度。

GATED RESIDUAL · QWEN CHOICE

四流分工：逐通道读 + 标量写，不再混流



仍把历史压进固定四路状态，但读门细到“分支 × 通道”，让一个子层可从不同分支抽取不同特征；写回只保留每分支一个标量，并完全移除 H_{res} 。

先看伪代码：差别发生在 read、write，还是 state

```

# 1) 普通 PreNorm: state 只有一条流；无法单独访问某个早期输出
def prenorm_step(R, F):
    # R: [d]
    x = rms_norm(R)          # fixed read
    y = F(x)
    R = R + y                # fixed write, coefficient = 1
    return R

# 2) mHC: state 是固定 n_r 条流；读写为分支标量，H_res 负责旧状态跨分支混合
def mhc_step(R, F):
    # R: [n_r, d]
    Z = group_rms_norm(R)
    h_mix, h_write, H_res = router(Z)  # [n_r], [n_r], [n_r, n_r]
    H_res = sinkhorn(H_res)            # nonnegative; each row/column sums to 1
    x = sum(h_mix[i] * R[i] for i in branches)
    y = F(rms_norm(x))
    R = H_res @ R + h_write[:, None] * y  # mix old state, then broadcast-write y
    return R
  
```

```

# 3) Full AttnRes: state 是全部历史子层输出; 沿 depth 做 softmax 检索
def attnres_step(history, layer_id, F):      # history: [0..layer_id-1, d]
    q = learned_query[layer_id]             # 每层一个参数向量, 不是 token Q 投影
    score = einsum(q, rms_norm(history))     # 每个 token、每个历史输出一个标量
    alpha = softmax(score, dim="depth")
    x = sum(alpha[i] * history[i] for i in depth)
    y = F(x)
    history.append(y)                       # 状态随深度增长
    return history

# 4) GR: state 固定为 4 条流; 逐通道读, 标量写, 无 H_res
def gr_step(R, F):                          # R: [4, d]
    Z = per_branch_rms_norm(R)
    G = sigmoid(up(silu(down(flatten(Z))))) # [4,d], bottleneck rank = d/8
    x = mean(G * Z, dim="branch")           # channel-wise read; 不是 softmax
    y = F(x)
    s = 2 * sigmoid(write_proj(flatten(Z))) # [4], each value in (0,2)
    R = R + s[:, None] * y                  # 各分支保留自身历史, 只接收新输出
    return R

```

伪代码省略 batch/token 维、静态项和实现细节, 目的是对齐四种机制的状态组织与读写接口。mHC 的具体参数化比这里更完整; Full AttnRes 的 key/value 都是早期子层输出, query 是每层一个可学习向量。

1. 普通残差：一条“不断做加法”的公共总线

$$\begin{aligned}
 x^{(\ell)} &= R^{(\ell)} \\
 y^{(\ell)} &= F^{(\ell)}(\text{Norm}(x^{(\ell)})) \\
 R^{(\ell+1)} &= R^{(\ell)} + y^{(\ell)} = R^{(0)} + \sum_{u \leq \ell} y^{(u)}
 \end{aligned}$$

普通 residual 的优点非常具体: 每个 token 只维护一个 d 维状态; 读写几乎不需要额外路由计算; 加法中的 identity term 为信息和梯度提供直接路径。它的问题也来自同一个公式: 所有历史输出都以固定权重 1 累加。第 20 个子层看到的是一个总和, 而不是一个带标签的历史列表; 如果它只需要第 2 层留下的某项特征, 只能尝试从混合后的向量中重新解码。

因此所谓“早期信号被稀释”不是早期输出凭空消失, 而是它在总状态中的相对占比随新增输出下降, 并且后层没有独立选择系数。后续子层若想增强自己的影响, 还可能倾向产生更大的输出, 进一步推动 residual magnitude 随深度增长。

2. mHC: 固定多分支状态, 但增加一次分支间重路由

$$\begin{aligned}
 x^{(\ell)} &= H_{\text{mix}}^T R^{(\ell)} \\
 y^{(\ell)} &= F^{(\ell)}(\text{Norm}(x^{(\ell)})) \\
 R^{(\ell+1)} &= H_{\text{res}} R^{(\ell)} + H_{\text{combine}} y^{(\ell)T}
 \end{aligned}$$

HC/mHC 把一条 d 维流扩为 n_r 条。 H_{mix} 决定当前子层从每条分支读多少， H_{combine} 决定新输出写进各分支多少， H_{res} 则在进入下一子层前重新混合旧分支。三者都可由当前 residual state 动态预测，所以不同 token 可以走不同的跨层路线。

关键是 H_{mix} 和 H_{combine} 在标准 HC/mHC 中主要是**每分支一个标量**：选中一条分支时，整条 d 维向量一起缩放。额外的 $n_r \times n_r$ 矩阵 H_{res} 提供分支间交换能力。mHC 用 sigmoid，并把 H_{res} 投影到双随机矩阵集合——元素非负、每行和每列均为 1——使旧状态的更新更接近受约束的“重新分配”，而不是任意放大；配合 identity 初始化，可保留稳定的直通路径。

mHC 不是“保留所有历史层”

它与 GR 一样，只维护固定 n_r 条状态。某条分支仍是许多早期输出的累加与混合，只是多分支和动态路由降低了单一总和的瓶颈。它增加的是固定宽度的跨层容量，不是可随机访问的完整深度缓存。

3. Attention Residual: 把“层深”当作 attention 的序列轴

$$\begin{aligned} v_0 &= \text{embedding}, \quad v_i = f_i(h_i) \\ \alpha_{i \rightarrow \ell} &= \text{softmax}_{i < \ell} (w_\ell^\top \text{RMSNorm}(v_i)) \\ h_\ell &= \sum_{i=0 \dots \ell-1} \alpha_{i \rightarrow \ell} v_i \end{aligned}$$

Full AttnRes 不再递归维护一个 residual sum，而是保存 embedding 与每个早期子层的输出。第 ℓ 个子层用一个可学习的伪 query w_ℓ 对这些输出计算 softmax，再形成自己的输入。query 对该层固定，但 key 来自每个 token 的历史表示，因此深度权重仍然随输入而变。

它与序列 attention 的类比很直观：token attention 在时间轴上检索早期 token，AttnRes 在深度轴上检索早期 layer output。Full 版可直接回到某个早期输出，代价是单个 token 需保留 $O(Ld)$ 历史，全部层的深度 attention 算术量为 $O(L^2d)$ ；在 activation recomputation 和 pipeline parallelism 下，这些历史还需要持续保存与跨 stage 传输。

Block AttnRes 将每 S 个子层输出先求和成一个 block representation，再在 block 间 attention，使状态与通信从 $O(Ld)$ 降到 $O((L/S)d)$ 。代价是 block 内输出再次失去身份：softmax 只能选择“这一组层的总和”，无法区分组内哪一层真正有用。

4. GR: 用四个压缩记忆槽，换取更细的读取粒度

$$\begin{aligned} \hat{R}_i &= \text{RMSNorm}(R_i; r_i) \\ G &= \text{unvec}(\sigma(W_u \text{SiLU}((1/n_r)W_d \text{vec}(\hat{R})))) \in \mathbb{R}^{n_r \times d} \\ x &= (1/n_r) \sum_i G_i \odot \hat{R}_i \\ s &= 2 \sigma((1/n_r)W_w \text{vec}(\hat{R})), \quad R'_i = R_i + s_i y \end{aligned}$$

GR 延续 widened residual 的固定多流结构，但把表达力从 H_{res} 转移到 read。它先分别归一化四条分支，再由全部 $4d$ 状态通过 $d/8$ 的低秩瓶颈，生成 $4 \times d$ 个 sigmoid gate。于是同一个子层可以从分支 1 取“语法相关通道”、从分支 3 取“长程实体相关通道”，而不是像 mHC 那样对整条分支使用同一个缩放系数。

这里的 G 不是沿分支 softmax：四条分支的权重不要求和为 1，每个通道可以同时开放多条分支，也可以全部压低。写端则只生成四个 (0,2) 标量，把同一个子层输出以不同强度加回各分支。报告消融发现逐通道 read 有收益，逐通道 write 几乎没有，因此最终设计把容量花在读取而非写入。

GROUP RMSNORM

四条分支分别归一化

每条分支拥有独立 gain γ ；GR read 本身已完成归一化，因此取代普通 block pre-norm，而非在它前面再叠一层。

LOW-RANK ROUTER

低秩预测，逐通道执行

路由器从 4d 压到 d/8 再升到 4d。当前 d=2560，瓶颈 rank=320；输出 gate 仍覆盖每个分支和通道。

NO HRES

分支不互相改写

每条分支只保留自己的历史并接收新 y。移除 H_{res} 少一次完整 residual-state 读取，也让路径贡献可精确分解。

一个具体例子：当前层需要“长期实体 + 最近句法”

需求 / 设计	普通残差	mHC	Full AttnRes	GR
状态中有什么	所有旧输出的一个总和	固定 n_r 条累加并动态混合的状态	每个早期子层的独立输出	固定 4 条彼此独立的累加状态
如何取长期实体	只能从总和中解码	提高保存该信息的整条分支权重	直接提高产生该表示的早期层权重	只打开长期分支中的实体相关通道
如何同时取最近句法	同一总和已包含，但不可独立调权	再提高另一分支；整条向量一起进入	同时 attention 到最近层输出	从局部分支打开另一组通道
付出的代价	最低；但选择能力最弱	n_r 倍状态流量 + H_{res} 混合	历史保存、深度 attention 与 pipeline 通信随 L 增长	4 倍状态流量 + 低秩 gate；无 H_{res}

放在一张表里：能力与成本分别来自哪里

设计	持久状态 (每 token)	选择粒度	旧状态如何演化	主要优势	主要成本 / 局限
Pre-norm residual	$O(d)$	无动态选择；所有历史权重为 1	单流累加	最省显存与带宽；identity path 简洁	早期输出不可单独取回；相对贡献随深度稀释
mHC, n_r branches	$O(n_r d)$	读写通常为每分支标量	H_{res} 动态跨分支混合	固定成本下扩大跨层容量；路由随输入变化	标量读不能按通道组合； H_{res} 增加整状态读取与约束
Full AttnRes	$O(Ld)$	每个历史子层一个 softmax 标量	历史输出独立保存并追加	可直接选择具体早期输出；选择空间最完整	全网 $O(L^2 d)$ 深度 attention；大规模重计算/流水并行下保存与通信昂贵
Block AttnRes, block size S	$O((L/S)d)$	每个 block 一个 softmax 标量	block 内先固定求和	保留部分深度检索，降低状态与通信	组内层身份被压缩；S 越大信息损失越明显
GR, $n_r=4$	$O(4d)$, 不随 L 增长	读为每分支 $\times$ 每通道；写为每分支标量	无分支混合，各自累加	固定状态下组合不同分支的不同通道；路径清晰	仍是压缩记忆而非完整历史；四路 residual 带宽是核心开销

复杂度只描述残差机制自身的持久状态与深度聚合，不代表整个 Transformer 的总 FLOPs。Full AttnRes 在普通、不做 activation recomputation 的训练中可复用本就保留的反向激活；在大规模重计算与 pipeline parallelism 下，额外保存和通信才成为主要问题。

Qwen 的直接消融：GR 相对 mHC，收益来自哪里

25B-A3B, 560B tokens, 4 branches	Loss	9 项平均	相对上一行的关键信号
Pre-norm	1.617	50.91	单流基线
mHC static	1.596	52.49	只扩宽与静态读写: loss -0.021, 平均 +1.58
mHC dynamic	1.594	54.47	动态路由: loss 只再降 0.002, 平均却再升 1.98
GR	1.590	54.66	逐通道读 + 移除 H_{res} : 质量相当略优, 状态读取更少

这组实验说明两件事。第一，提升并不全来自复杂门控：把 residual 从一条扩成四条、即使只使用静态算子，也贡献了主要 loss 降幅。第二，动态路由的价值更容易体现在 benchmark 而非训练 loss；若只看 0.002 的 loss 差，会低估 static→dynamic 带来的 1.98 分平均提升。

Qwen 的直接消融：AttnRes 的 loss 更低，为何最终仍选 GR

28 layers / 56 sublayers	原设计 Loss	+ GatedNorm	怎么读
Pre-norm	1.789	1.787	GN 改善 0.002
Block AttnRes, S=4	1.773	1.768	四个子层先求和; GN 改善 0.005
Block AttnRes, S=2	1.770	1.766	压缩更轻, 优于 S=4
Full AttnRes	1.762	1.758	本表最低 loss, 但状态/通信随深度增长
GR, $n_r=4$	—	1.762	与未加 GN 的 Full AttnRes 持平; 固定四路状态

这张表不能解读成“GR 在纯 loss 上击败 AttnRes”：Full AttnRes + GatedNorm 的 1.758 是最低值，GR 为 1.762。结合最终采用 GR 的架构与两类方案的系统成本看，实际取舍相当于用 0.004 loss 换固定 O(4d) 状态和更简单的流水并行路径。48 层的另一组实验中，Block AttnRes S=4 为 1.711，GR 为 1.707；报告没有在该设置给出 Full AttnRes 数字。

两张表不能横向硬排

mHC/GR 的 Table 5 是 25B-A3B、560B tokens，并报告 9 项 benchmark；AttnRes 的 Table 6 是 28 层 residual-design loss 实验。它们分别回答“GR 相对 mHC 的质量与路由取舍”和“固定状态 GR 相对深度 attention 的 loss/效率取舍”，不是同一排行榜。

四条分支最终学到了什么

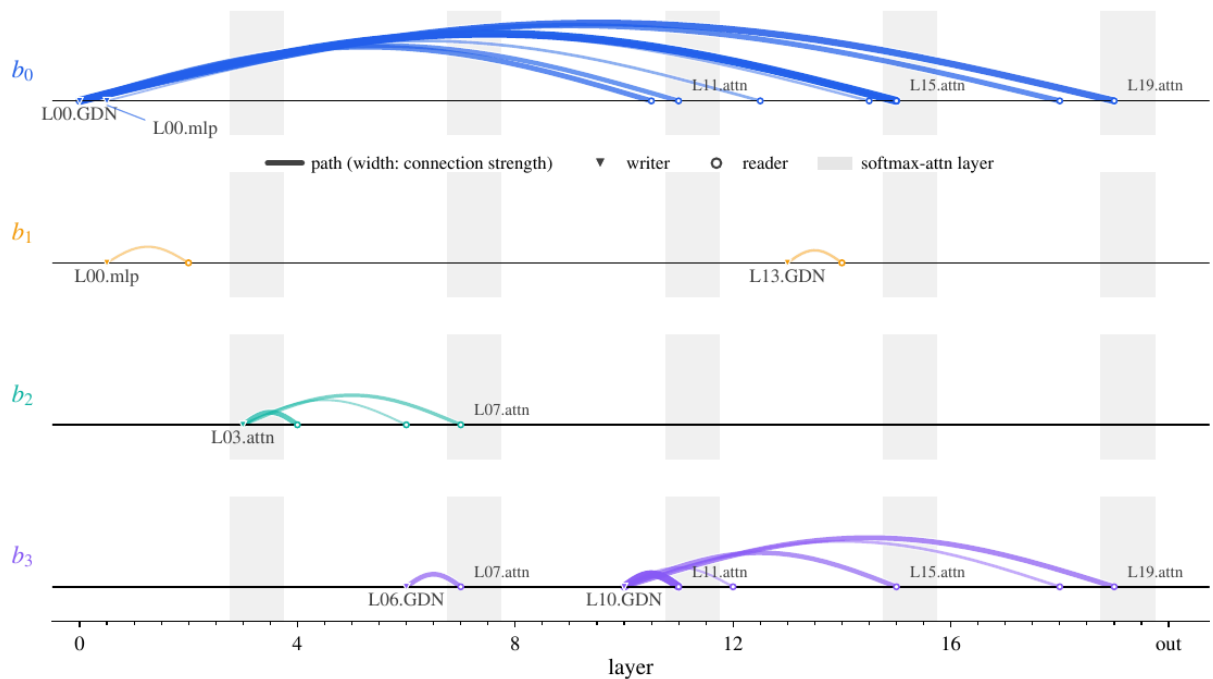


Figure 7: **Cross-layer paths added by GR.** Each row corresponds to one residual branch; a connection runs from the sublayer that wrote into that branch to a later sublayer that reads it back. Vertical position encodes the number of layers skipped; line width and opacity encode Δ_{uv} (Eq. (37)), the additional share of the reader’s input supplied by this writer compared to a single residual stream. Shaded regions denote softmax-attention layers, every fourth in this hybrid; the rest are GDN, and sublayers are named accordingly (L00.GDN, L03.attn, L00.mlp). Readers are named where they are softmax-attention sublayers, which is where the long-range paths land. One branch carries long-range paths (all connections on b_0

论文 Fig.7。在 20 层模型中，GR 学到一条典型 skip ≈ 10.9 层的长程分支，另外三条分支的典型 skip $\approx 3.4 - 3.9$ ；长程路径主要把早期 GDN 输出送到周期性的全局 attention 层。

路径分解给出比“残差更宽所以更强”更具体的机制：Layer 0 GDN $\rightarrow$ Layer 15 attention 的输入份额从 0.020 升到 0.138；Layer 10 GDN $\rightarrow$ Layer 11 attention 的增量也达到 0.117。总体上，adjacent path 与 skip > 12 的份额上升，中距离 2 - 12 层路径下降。GR 没有简单增加平均跨层距离，而是把容量重新分配给少数短程和长程关键通路。

一句话总结：普通 residual 是“一个不断覆盖语义的总和”；mHC 是“固定多槽记忆 + 标量路由 + 槽间重排”；AttnRes 是“保留完整层历史再做深度检索”；GR 则是“固定四槽记忆 + 逐通道读取”，用比 mHC 更细的 read 换掉 H_{res} ，并用比 AttnRes 更强的历史压缩换取固定系统成本。

推理侧为什么没有采用 Top-2 分支读

训练后可观察到写入常由两条分支主导，于是团队试过只读 gate 最大的两条分支。预训练 loss 与 benchmark 几乎不变，但后训练质量明显下降，最终放弃。实际优化选择是：将 residual state 保存为 FP8，使相对 BF16 的搬运字节减半；再把 read/write 各自融合成一个 kernel，并把 group RMSNorm 折进 read。

最终工程判断

GR 的优势不是“残差计算更少”——四路状态本身会增加带宽——而是它的成本不随网络深度增长，并且移除了 mHC 的 H_{res} 整状态重读。FP8 state 与 fused read/write 针对的正是这个带宽瓶颈。

06 · N-GRAM EMBEDDING

把“参数容量”从专家计算搬到确定性查表

MoE 用 router 选择要计算的专家；n-gram memory 用最近的局部 token 片段直接生成地址，查出向量并加到 token representation。前者增加条件计算，后者增加条件记忆，两者是正交的稀疏化轴。



为什么只放 Layer 2

固定 n-gram 参数预算时，单层位置从浅到深都能降低 loss，多层分摊没有一致收益。Layer 2 的平均分最高（47.94，基线 45.44），并允许在 Layer 1 计算期间预取查表结果。选择 Layer 2 因而同时满足质量与系统重叠。

实验问题	结果	设计决策
相同总参数：减少专家、增加 n-gram	loss 在 $10\times$ vocab 最低，但下游不稳定，未形成对 MoE-only 的清晰优势	不把 n-gram 当成专家的等价替代
保持 MoE 不变，额外增加 n-gram	$20\times\rightarrow200\times$ vocab 时 loss 1.553 $\rightarrow$ 1.526 单调下降；多数 benchmark 饱和或波动	作为额外 memory capacity 扩容
中文 benchmark	C-Eval 71.75 $\rightarrow$ 74.94, CMMLU 72.29 $\rightarrow$ 73.24, 随 vocab 较一致上升	局部词片段查表对中文组合模式尤其有效

51B 参数不意味着 51B 都在每 token 上计算

embedding table 是稀疏访问：每个 token 只命中少量行。主要代价是容量、地址计算和内存搬运，而不是对 51B 权重做矩阵乘。确定性地址还使下一批查表可预知，因此适合 host offload 与 prefetch。

技术来源方面，报告把该模块放在 N-Grammer、Gemma 3n、DeepEmbed、Conditional Memory/Engram、Scaling Embeddings 等 lookup-memory 路线中。最终设计与 Engram 在“n-gram 地址、离卡存储、预取”的思想上接近；报告将这些工作并列引用，未指定单一直接祖先。

07 · OPTIMIZER & SYSTEM

Muon 不只是换更新公式，还重做了参数归属与分布式执行

Muon 对矩阵动量做 Newton – Schulz 正交化。真正困难的是：Megatron 中一个逻辑矩阵可能被 TP 切开，而不同矩阵的 NS FLOPs 随短边近似三次增长，按参数元素平均切 DP 会产生严重 straggler。Qwen 通过 Canzona 解耦“物理存放位置”和“谁

负责算 optimizer”。

哪些参数用 Muon，哪些不用

参数类别	优化器	原因
attention Q/K/V/O; GDN 输入/输出; MoE fc1/fc2; n-gram K/V projection	Muon	二维权重确实扮演共享结构的线性映射
input embedding、LM head	AdamW	不是普通隐层矩阵更新问题
MoE router	AdamW	Muon 加剧早期波动; 专家 logit 维度更接近相互独立输出
GR 两个细长 low-rank projection	AdamW	极长宽比下消融表现更好
attention/GDN output gate、GDN α/β 向量	AdamW	gate 不优于 AdamW; 标量/向量不适合矩阵正交化
n-gram table	Adam, 无 weight decay	稀疏 lookup table

Muon 细节

- Nesterov momentum $\mu=0.95$; 更新按矩阵形状缩放, 使 update RMS 不依赖 shape。
- 使用 Polar Express 的逐步系数; Newton - Schulz 迭代 8 步。更多迭代不仅正交化更准确, 也降低 stress test 的 gradient spike。
- QKV、SwiGLU fc1、GDN input 在 Megatron 里是 fused matrix, 但语义上由独立算子拼接。报告先按 head 或 gate/up 切分, 再各自 NS, 避免把无关奇异方向混在一起。

Canzona 的分布式路线



切分 fused 参数后, 一层会产生约百个小矩阵, optimizer step 容易从算力受限变成 kernel launch 受限。最终把整个 step 捕获进 CUDA Graph。这里的贡献重点是让 Muon 在 Megatron 3D parallelism 下保持数学等价、负载均衡和通信重叠, 不是更改 Muon 的目标函数。

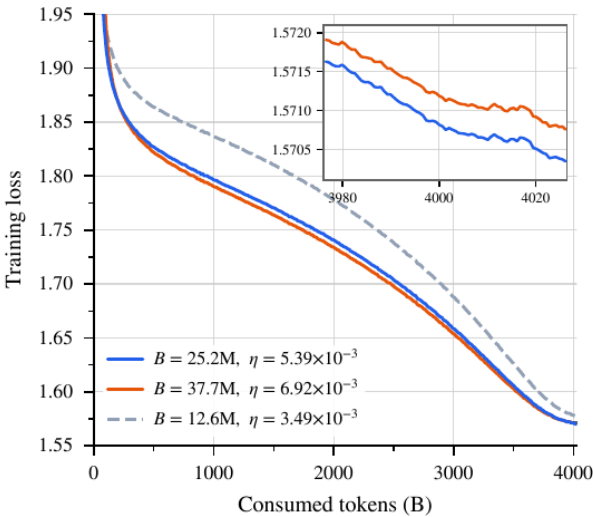
相关系统证据

独立 Canzona 论文在 Qwen3 系列、最高 256 GPU 的实验中报告相对基线 $1.57\times$ iteration speedup 与 $5.8\times$ optimizer-step latency 降低。Qwen3.8 报告说明采用该框架, 但没有披露本次 125B 生产训练的完整端到端吞吐或 GPU 数。

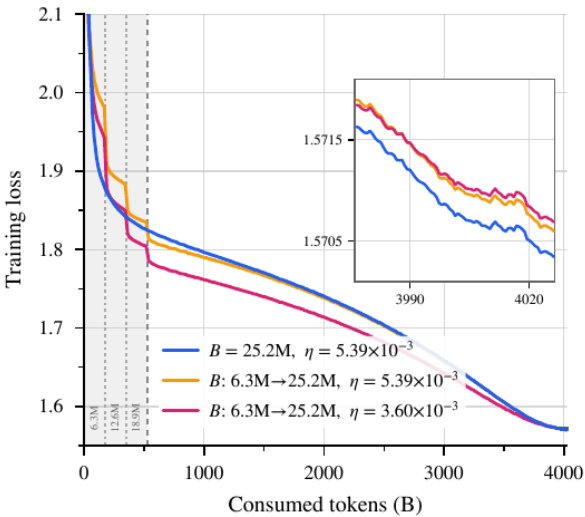
08 · HYPERPARAMETER SCALING

新架构 + Muon 改变了 “最合适的 batch 与学习率”

旧 Qwen3.5 的 scaling law 不能直接平移。团队重新拟合 η 与 batch size: 新配方的近优区域整体向更大 batch、更高 learning rate 移动, 而且大 batch 的损失曲线在最优点上方很平, 在下方却陡峭。



(a) Constant batch size.



(b) Ramped batch size.

Figure 8: **Batch size on a 4T-token budget.** Training loss against consumed tokens for a 20-layer 10.8B-A0.89B MoE; each inset resolves the last 50B tokens. (a) Moving from the previous recipe ($B = 12.6\text{M}$) to the predicted optimum ($B = 25.2\text{M}$) is worth 7.2×10^{-3} , while a further step to $B = 37.7\text{M}$ leads to a minor degradation. The loss rises steeply below the prediction and remains nearly flat above it. (b) Reaching $B = 25.2\text{M}$ through a ramp instead performs no better than using this batch size from the start and takes 18.8% more optimizer steps.

learning rate (η) and batch size (B) (Qwen Team, 2024; Yang et al., 2025). The optimal learning rate and batch size depend on the model architecture and the optimizer, so a recipe that was optimal for the previous-generation model may become suboptimal once both change. Under our previous Owen3.5 hv-

论文 Fig.8。20-layer 10.8B-A0.89B、4T-token 预算：B=25.2M 最优；37.7M 几乎同水平；旧 12.6M 明显更差。batch warmup 不能超过直接从 25.2M 开始。

验证	旧配方	新预测	观察
小模型、超长 token budget	B=12.6M	B=25.2M	最后 20B token loss: 1.5774 → 1.5702
batch warmup	6.3M 分段升到 25.2M	固定 25.2M	warmup 略差，并多 18.8% optimizer steps
大模型、419B-token	B=4.2M, $\eta=6.8\text{e-}4$	B=8.4M, $\eta=1.76\text{e-}3$	旧配方最终 loss 高 0.0078; benchmark 平均 56.41 vs 60.55

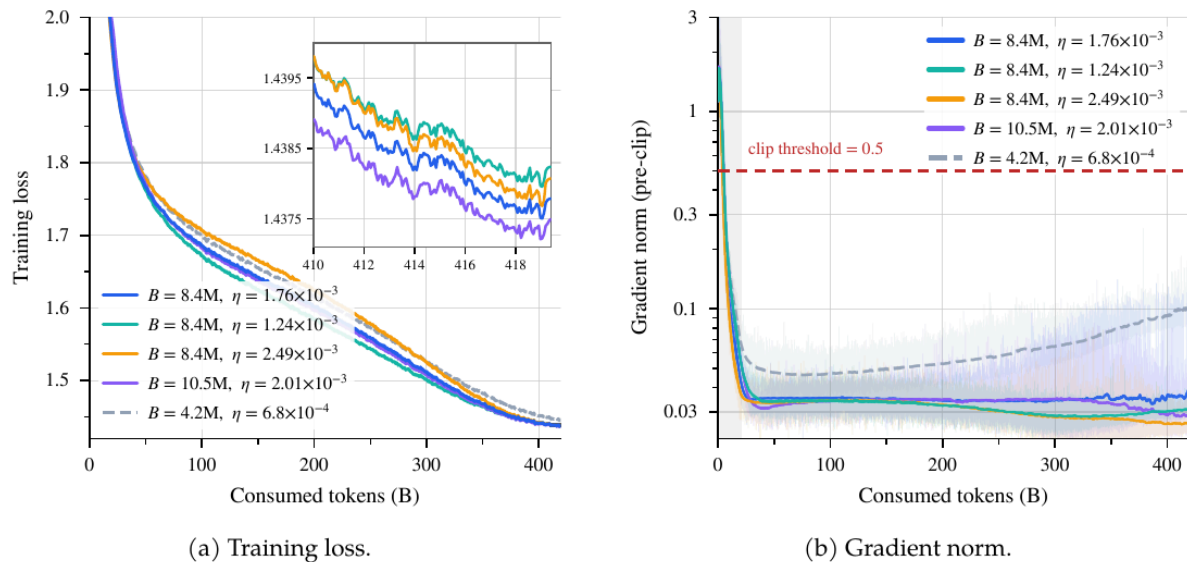


Figure 9: **Learning rate at a larger model scale.** Five runs of a 48-layer MoE on a 419B-token budget: the predicted optimum ($B = 8.4M, \eta = 1.76 \times 10^{-3}$), η divided and multiplied by $\sqrt{2}$, a 25% larger batch with its matched η , and the previous recipe (dashed). (a) The inset covers the last 10B tokens. The loss differences among the predicted optimum and the three nearby settings are near the noise level, so the predicted optimum sits at the bottom of a flat bowl. (b) Pre-clip gradient norm on a log scale, with individual steps faint behind a moving average, the warmup shaded and the clip threshold dashed. After warmup, the pre-clip gradient norms of all runs near the predicted optimum remain below 50% of the clipping threshold, including the run at $\sqrt{2}$ times the predicted learning rate.

Learning Rate at a Larger Model Scale. The learning-rate prediction is tested on a much larger 48 layers 156B-A7B MoE, over a 419B-token budget that is again the same for every run. We evaluate the predicted optimum together with learning rates scaled by $1/\sqrt{2}$ and $\sqrt{2}$, a 25% larger batch with its matched learning rate, and the previous hyperparameter recipe.

论文 Fig.9. 48-layer 156B-A7B: 预测最优点与 $\pm\sqrt{2}$ learning-rate、+25% batch 的四组结果仅差约 7×10^{-4} , 说明新最优点位于较宽的平坦谷底; 旧配方显著落后。

为什么取消 batch warmup

在 Muon + MoE 设置下, 小 batch 带来的梯度噪声和每专家 token 多样性不足, 比“大 batch 早期可能浪费样本”更严重。warmup 虽短期积累更多 optimizer steps, 但最终被固定大 batch 反超。这里的结论来自特定架构与优化器, 不是对所有 LLM 训练的通用否定。

09 · TRAINING STABILITY

稳定性不是“没炸过”, 而是用高学习率主动把故障放大

大模型生产训练的偶发 spike 很难用小实验观察。报告采用 stress test: 在 28-layer MoE 上保持 $2\times$ 、 $4\times$ 最优学习率不衰减, 让本应在超长生产训练中才出现的不稳定性在 10 万步内暴露。

2 × LR**Muon 抵抗早期 spike**

AdamW Qwen3.5 为 4.3 spikes/10k; 两种 Muon 配置约 0.2/10k。

4 × LR**GR 拉开类别差异**

AdamW 183 spikes/10k; Muon+GR 为 0, 且 Muon 组都未跨 0.5 clipping threshold。

GATE ISOLATION**控制变量确认 gate 作用**

AdamW、结构、数据顺序相同, 打开 GatedNorm 后 spike 32.0→3.2/10k, threshold crossing 256→20。

(Fig. 10). The AdamW run spikes on 183 per 10k steps and crosses the clipping threshold on 213 of 19,932 steps, engaging the clipper continuously. In contrast, both Muon runs never cross the clipping threshold, and the configuration with the gated residual records zero loss spikes. Under equal stress, the new architecture and optimizer combination is substantially more stable.

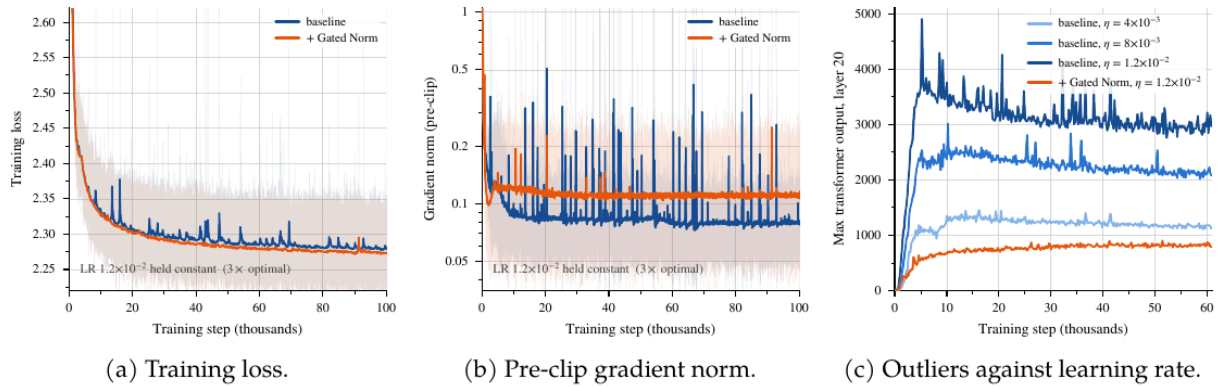


Figure 12: Isolating the effect of the gate. GatedNorm off and on, with the AdamW optimizer, structure and data order held fixed. (a) and (b) are the pair at $3\times$ the optimal learning rate; bold lines are a moving average over the faint per-step trace. (c) adds the ungated baseline at $1\times$ and $2\times$ the optimal rate.

论文 Fig.12。显式 multiplicative gate 给网络一个直接的重缩放通道；在无 gate 模型中，学习率升高会伴随 activation outlier 快速增长。

生产规模前 276B token 的同数据对照

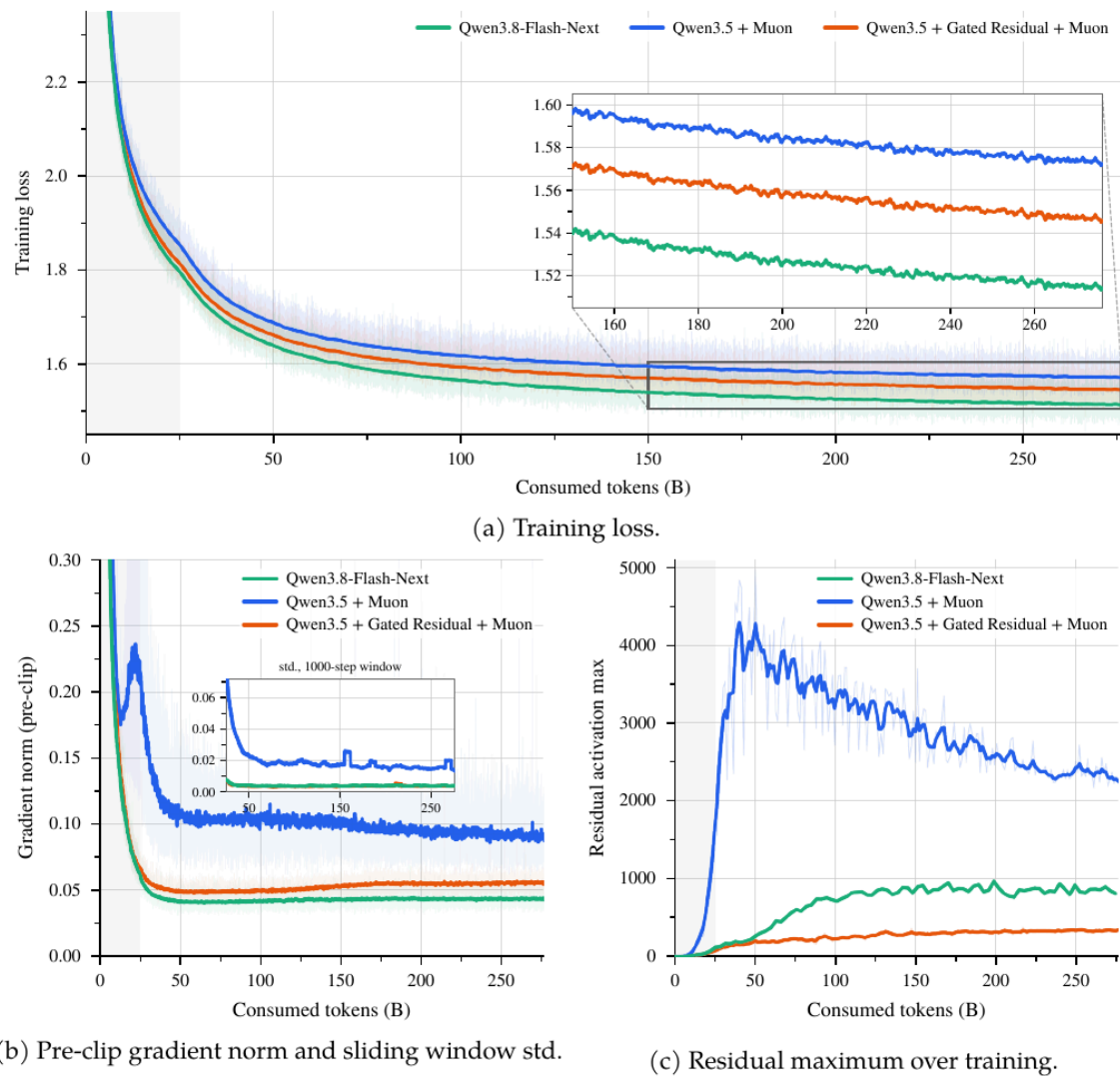


Figure 13: **The early phase of Qwen3.8-Flash-Next, at the shipped learning rate.** The first 276B tokens of three runs that share data order, learning-rate schedule, and optimizer: Qwen3.5 with Muon, the same plus the gated residual, and Qwen3.8-Flash-Next. The shaded band corresponds to the learning-rate warmup. The inset in (a) resolves the last 126B tokens of the window; the inset in (b) is the standard deviation of the gradient norm inside a rolling 1000-step window.

论文 Fig.13。三组运行共享数据顺序、学习率 schedule 与 Muon: Qwen3.5 backbone; 加 GR; 完整 Flash-Next。GR 将 276B 时 loss 降 0.026, 完整架构再降 0.032。

- Qwen3.5+Muon 的 median/p99.9 gradient norm 为 0.097/0.298; 加 GR 后 0.053/0.071; 完整 Flash-Next 为 0.043/0.066。
- 两组 gated run 的 1000-step rolling std 低 4.3 - 4.7 $\times$; 只有无 gate 的 Muon run 跨过 clipping threshold。
- GR 显著降低各层 residual maximum。最终训练不依赖 qk-clip 或 SwiGLU-clip 这类显式 activation clipping。

机制解释

高学习率要求网络不断调节信号幅度。没有显式 gate 时, 网络可能通过放大 activation outlier 间接实现重缩放, 导致脆弱性; GR/GatedNorm 直接提供 bounded multiplicative path, 因此不必把幅度压力转移到异常激活。

10 · DISCLOSURE AUDIT

训练数据、算力、时长与部署：报告披露到哪里

这份报告的重点是架构实验，不是完整训练或部署白皮书。它披露了大量 ablation token budget、QSA CPT token 和相对成本，但没有给出主模型总训练 token 的绝对值、训练卡数、硬件拓扑、墙钟时长或 MFU。

问题	报告可确认	仍未披露
主模型训练数据量	相对 Qwen3.7-Plus 约 1/3 training tokens; QSA Stage 1≈2B、Stage 2≈200B; 生产对照展示前 276B	完整预训练 token 绝对量、数据构成、去重与语言比例
训练算力	相对训练 FLOPs 约 1/9; FlashQLA 在 NVIDIA GPU 上测试	GPU 型号、卡数、并行配置、MFU、总 GPU hours
训练时长	无	墙钟天数、失败/恢复成本
长上下文训练	QSA 在 256K CPT 中引入，共 9000 steps、约 202B token	完整 context curriculum 与 YaRN 训练细节
推理系统	QSA/FlashQLA/GR fused kernel; FP8 residual; host-memory n-gram prefetch; CUDA Graph optimizer	官方集群调度、KV cache 层级、PD disaggregation、端到端吞吐/TTFT

能否理解为“用了 vLLM / SGLang 就完成部署优化”

官方模型卡给出 SGLang、vLLM 和 TokenSpeed 的部署入口，这说明生态兼容已经完成；技术报告中的主要效率工作却发生在模型算子与训练系统层：FlashQLA、QSA sparse kernel、GR read/write fusion、FP8 residual、n-gram host prefetch、Canzona 和 CUDA Graph。它没有给出类似 Mooncake 的多级缓存、PD 分离或跨节点 serving 调度设计，因此不能据此声称完成了全栈在线服务优化。

报告中的 7.6× / 4.9× 不应外推到 API 性能

这两个数字是 1M context、指定 batch/chunk 条件下的 attention-module kernel latency，对比 paged GQA。端到端服务还受 MoE、视觉编码器、n-gram I/O、调度、batching、网络通信与采样影响。

11 · EVALUATION

论文证明的是 base-model 效率前沿，不是完整产品能力排名

Table 11 比较 Qwen3.8-Flash-Next-Base、Qwen3.8-27B-Base 与 Qwen3.7-Plus-Base。Flash-Next 在 14 项中全部超过 27B dense，并在 8/14 超过 397B-A17B 前代；其余差距最大 2.6 分。

- **Multilingual tasks:** MGSM (Sui et al., 2022) (8-shot, Co1), MMMLU (OpenAI, 2024) (5-shot), INCLUDE (Romanou et al., 2024) (5-shot).

Table 11: Comparison among the base models of Qwen3.8-Flash-Next, Qwen3.8-27B and Qwen3.7-Plus. The highest and second-best scores are shown in **bold** and underlined, respectively.

	Qwen3.8-Flash-Next-Base	Qwen3.8-27B-Base	Qwen3.7-Plus-Base
# Params	125B	27B	397B
# Activated Params	6B	27B	17B
# N-gram Embedding Params	51B	–	–
General Tasks			
MMLU	<u>90.36</u>	87.51	90.43
MMLU-Redux	<u>90.68</u>	87.26	91.47
MMLU-Pro	73.23	68.60	<u>70.90</u>
SuperGPQA	51.36	44.86	<u>48.42</u>
BBH	90.87	89.56	<u>89.41</u>
Math & STEM Tasks			
GPQA	<u>51.42</u>	45.01	51.52
GSM8K	93.29	<u>93.18</u>	92.95
MATH	<u>72.78</u>	60.54	74.38
Coding Tasks			
EvalPlus	78.76	76.05	<u>78.06</u>
MultiPL-E	<u>79.09</u>	74.50	81.68
SWEBench-Pretrain	50.99	41.66	<u>49.24</u>
Multilingual Tasks			
MGSM	89.33	<u>86.37</u>	85.42
MMMLU	84.86	<u>79.74</u>	<u>84.53</u>
INCLUDE	<u>78.40</u>	74.37	78.90

论文 Table 11。优势集中在 MMLU-Pro、SuperGPQA、BBH、EvalPlus、SWE-Bench-Pretrain、MGSM、MMMLU；Qwen3.7-Plus 在 MMLU-Redux、MATH、MultiPL-E、INCLUDE 等仍领先。

VS 27B DENSE

14 / 14 领先

说明 6B active 的稀疏模型在此训练量下建立了更高容量效率。

VS 397B-A17B

8 / 14 领先

以 1/3 active params、1/3 tokens 达到相近 base quality。

COST CLAIM

≈ 1/9 training FLOPs

主要由 active params 与 token 两个约 1/3 相乘得来，是近似相对口径。

如何看官方后训练 benchmark

官方模型卡另给出 DeepSWE、SWE-bench Pro、CoWorkBench、OSWorld、MathVision 等语言/视觉 agent 结果。这些数据证明公开 checkpoint 的产品能力，但并非本技术报告的同一组实验：部分是内部 benchmark，harness、上下文与 judge 也各不相同。本文把它们作为发布补充，不混入论文 Table 11 的架构因果论证。

哪些因果结论仍不完整

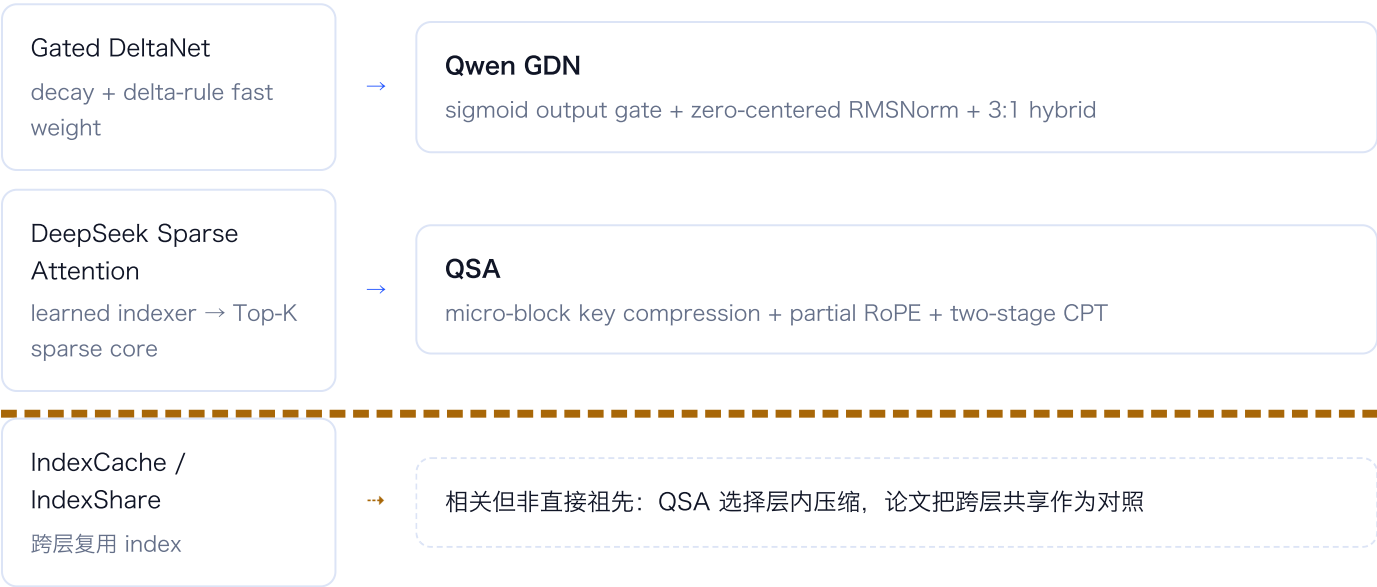
最终模型由 GDN/QSA、GR、n-gram、Muon 与新训练配方共同组成。报告提供大量中等规模单模块消融，但没有在最终 125B 模型上给出完整 factorial ablation；因此“总提升由每个模块分别贡献多少”不能从 Table 11 反推。

12 · TECHNICAL PROVENANCE

技术谱系：明确继承、再设计与相关路线

下图按证据强度描述技术关系。实线节点表示报告明确采用或组合；虚线节点表示同问题域的结构比较。相近不自动等于来源。

Token mixing



Residual



对跨层输出做 attention read

Memory & optimization



13 · EVIDENCE BOUNDARIES

证据边界与阅读结论



最终评价

Qwen3.8-Flash-Next 最突出的贡献不是发明一个全新基本算子，而是把四条已有研究线索推成一套可共同训练、可长上下文推理、可离卡扩容的生产候选架构。报告的说服力来自大规模消融与失败方案披露；其边界则是缺少完整数据/算力账本、最终模型模块级消融和端到端 serving 指标。

14 · PRIMARY SOURCES

一手资料与关键论文

01	Qwen3.8-Next Technical Report 本文主证据：架构、消融、训练稳定性与 base evaluation。
02	Qwen3.8-Flash-Next Official Blog 官方发布定位、256K/1M context 与 Qwen4 preview。
03	Official Model Card · config.json 模型结构参数、MTP、视觉编码器、benchmark 与部署入口。
04	Gated Delta Networks GDN 的直接结构来源。
05	DeepSeek-V3.2 / DeepSeek Sparse Attention QSA 明确跟随的 learned-indexer sparse-attention 路线。
06	FlashQLA · FLA Qwen GDN fused kernel 与对比基线。
07	AltUp · Hyper-Connections · mHC 残差流扩宽与动态 read/write 主线。
08	A Unified View of Attention and Residual Sinks GatedNorm 与显式重缩放机制。
09	Attention Residuals GR 的跨层读取比较对象。
10	N-Grammer · Conditional Memory / Engram · Scaling Embeddings n-gram lookup-memory 与 embedding scaling 背景。
11	Muon · Polar Express 矩阵优化器与 Newton - Schulz 系数方案。
12	Canzona 分布式矩阵优化器的异步、负载均衡执行框架。
13	GLM-5 MTP speculative step 之间复用 sparse Top-K index 的直接参考。

